CASE STUDY · SHOWCASE EDITION

Legacy SAS → CDP Migration

Volume 2

Diagnosis & Remediation

The diagnostic method, the full defect register and how the pipeline reached 475 of 475

WHAT THIS VOLUME COVERS

The derivation path used for every defect, a fault taxonomy, a fourteen-entry defect register with root cause, fix, verification and effect, the bridge-file specifications, the population progression from 344 to 475, and the change-control practices introduced along the way.

ABOUT THIS DOCUMENT

"Meridian Media" is a hypothetical organisation. All client names, data values, file names and code identifiers have been anonymised or generalised to protect client identity and intellectual property. Figures are illustrative and rounded; the method and sequence of work are described as executed.

Client Analytics · Project Documentation Set (3 volumes)

Contents

1. Diagnostic Method

Variables that fail silently cannot be fixed by reading code. Every defect in this engagement was taken through the same derivation path, and no code was changed until the path had produced evidence.

1.1 The derivation path

Step	Question	Evidence used
1. Symptom reported	What does the client or QC see?	Match-rate report; population census
2. Classify the fault	Absent, zero-filled or understated?	Log scan; value distribution
3. Read the emitted SQL	What did the engine actually send?	Run log; macro trace
4. Profile the source	Is the column populated? Do keys share a domain? Do filter values exist?	Read-only probe against the CDP
5. Branch	Does the source hold the data?	Steps 3–4

1.2 The branch after step 4

- **Source has the data → code fix.** Correct the shared generator, never the individual variable. Re-run on the test harness, confirm by value, then promote.
- **Source does not hold it in the expected form → data escalation.** Evidence the gap at source, confirm the variable populates in legacy, and take it to the business or platform team with a proposed rule or mapping and an owner.

> **PRINCIPLE**
> Profile the source before changing the code. In this engagement, three proposed rebuilds — a lookup table, a parse rule and a key change — were replaced by one-line corrections because profiling showed each fault sat upstream of the logic.

1.3 Read-only probes

Every hypothesis was expressed as a SELECT-only probe. Typical probes:

- **Column profile** — row count, NULL count and distinct count for a candidate column.
- **Predicate comparison** — the same aggregate computed with the emitted predicate and with the corrected one; the difference sizes the defect.
- **Key overlap** — the share of values two candidate keys have in common.
- **Route comparison** — the result of alternative join paths for the same business concept.

2. Fault Taxonomy

Three faults were repeatedly reported in the same words — "the variable isn't right" — and needed different tests.

Class	Definition	Where to look	Example here
Absent	The variable never built; the column is missing from output	The run log: the query failed and the cascade dropped the column	13 interest variables (D03)
Zero-filled	Built, but matched nothing	Source profile: the join or filter cannot match	27 affinity variables (D05)
Understated	Built and matched, but rows were silently excluded	Value comparison against source or legacy	Magazine revenue (D03)
Environmental	Logic correct; run state or deployment wrong	Run state, redirected log, file integrity	D08, D09, D10

3. Defect Register

Fourteen defects were recorded. Identifiers, column names and values are generalised in this edition.

D01 — Unresolved legacy column names

Attribute	Detail
Fault class	Absent (cascade)
Symptom	First CDP run produced ~5,000 errors; 131 of 475 variables missing.
How it was found	Emitted SQL contained legacy column names that do not exist on the CDP core objects.
Root cause	The CDP renamed and re-typed core columns. Recipes reference names directly.
Fix	Per-table column mapper in a new bridge file; unknown names pass through unchanged. Join builders overridden so both sides resolve through the mapper.
Where	compat_bridge.sas (new)
Verification	Emitted SQL inspected for every builder; no unresolved names remained.
Effect	Naming errors removed. The error count stayed high because each failing query also produced a cascade (see D03).

WHY ~5,000 ERRORS

Every failing variable query produced four to five follow-on errors — staging table missing, merge failed, rename failed. The headline count therefore overstated the number of distinct defects by roughly five times.

D02 — Incorrect key rewrite on marketing activity

Attribute	Detail
Fault class	Absent
Symptom	About 79 promotion-history variables missing.
How it was found	Log showed queries referencing a customer-key column that the marketing-activity view does not carry.
Root cause	An early mapping rewrote the customer key to a 'current key' column assumed to exist on every view.
Fix	Rewrite removed for that view; the view's native key used.
Where	compat_bridge.sas
Verification	Column profile of the view; harness re-run.
Effect	+79 variables (344 → 423).

D03 — NULL-dropping exclusion filters

Attribute	Detail
Fault class	Understated, then Absent
Symptom	Magazine revenue ~35% below legacy on a sampled audience; later, 13 interest variables missing.
How it was found	Predicate comparison: revenue summed with the emitted ≠ filter versus a NULL-retaining filter.

Attribute	Detail
Root cause	Under ANSI three-valued logic, NULL ≠ 'X' is UNKNOWN, so rows with a NULL filter column are dropped. On Oracle these columns were always populated; on the CDP many were NULL.
Fix	Exclusion branch in all three builders changed to (col IS NULL OR col ≠ val). An interim fix using coalesce(col,'') worked on text columns but failed on numeric ones, removing 13 variables until replaced.
Where	sql_engine.sas — exclusion branch, three builders, six lines
Verification	Harness run: type-failure cascade cleared; ~35% of sampled revenue restored; 280 sampled customers exceeded the value the old predicate could return.
Effect	Errors ~5,000 → 3. +13 variables (423 → 436). Values corrected across 118 variables.

NOTE

The three residual errors were workspace-disk exhaustion, not logic. They were cleared operationally.

D04 — Email-tenure date from the wrong field

Attribute	Detail
Fault class	Understated (wrong values)
Symptom	Tenure date present, but every value recent.
How it was found	Distribution of the source field: values clustered on a handful of migration-load dates.
Root cause	The advised CDP field records when a record was loaded, not when the address was captured.
Fix	Purpose-built recipe: coalesce(original created date, platform load timestamp). Output converted to calendar date.
Where	compat_bridge.sas (dedicated recipe)
Verification	Year-by-year distribution; legacy dates restored back a decade.
Effect	+1 variable (436 → 437). ~50% fill on the full audience, consistent with legacy.

GOVERNANCE

Agreed with the platform build team and the client's modelling lead before implementation (Volume 3, decision register).

D05 — Product mapping pointed at an empty column

Attribute	Detail
Fault class	Zero-filled
Symptom	27 affinity (content-class) variables zero for every customer.
How it was found	Column profile: the mapped product-code column held NULL on every row — zero distinct values.
Root cause	The mapper pointed at a column that migrated empty. A differently named product-code column was fully populated.
Fix	One mapping line changed.
Where	compat_bridge.sas
Verification	Join match count: zero before, billions after. Harness census.

Attribute	Detail
Effect	+27 variables (437 → 464).

TWO FIRST ANSWERS THAT DID NOT SURVIVE

The first explanation was ambiguity — one product code spanning titles with different content classes — and the proposed remedy was a title-level lookup and parse rule. A second proposal to switch the title key would have broken 25 working variables; testing showed zero matches against tens of millions for the existing key. Both were stopped by profiling.

D06 — Date-type semantics

Attribute	Detail
Fault class	Understated (wrong values)
Symptom	A days-since-shipment variable carried the build date itself.
How it was found	Spotted by the client in output; confirmed by inspecting returned column types.
Root cause	Offset arithmetic extracts the date part of a DATETIME. Some CDP columns return DATE, so the conversion produced a wrong offset without error.
Fix	Type detection at run time; guarded conversion — date-part extraction only where valid.
Where	date_bridge.sas (new)
Verification	Recency family compared against legacy after normalisation.
Effect	All date variables carry a consistent, verifiable offset.

D07 — Key-domain mismatch in a new recipe

Attribute	Detail
Fault class	Environmental / logic
Symptom	Output rows ~50% above the requested audience.
How it was found	Row-count check; the appended rows carried only the new variable.
Root cause	The recipe grouped on the platform's profile key but labelled it as the customer key. The two keys share no values, so the merge appended instead of joining.
Fix	Join on the platform key; select and group on the customer key.
Where	compat_bridge.sas (dedicated recipe)
Verification	Row count equals audience exactly.
Effect	Output integrity restored.

D08 — Leftover staging table

Attribute	Detail
Fault class	Environmental
Symptom	Run completed 'successfully' with zero rows.
How it was found	The redirected run log showed the audience CREATE had failed.
Root cause	An aborted run left its permanent audience table in the shared schema; the drop runs only on clean completion.

Attribute	Detail
Fix	Existence guard drops any stale table before rebuild.
Where	orchestrator.sas
Verification	Deliberately aborted run followed by a clean run.
Effect	Silent empty runs eliminated.

D09 — Bridge file overwritten

Attribute	Detail
Fault class	Environmental
Symptom	Thousands of 'macro not resolved' warnings; hundreds of variables lost.
How it was found	Fresh-session check: bridge macros did not exist.
Root cause	A one-macro patch was saved over the full bridge file.
Fix	File restored from source; change control introduced (section 6).
Where	Process
Verification	Fresh-session compile check returns every expected macro.
Effect	Recurrence prevented.

D10 — Unseen reference code

Attribute	Detail
Fault class	Environmental
Symptom	Run stopped before building any variable.
How it was found	Integrity-check message in the redirected log naming the unknown code.
Root cause	A new product code was loaded to the CDP after go-live.
Fix	Code placed in a neutral group read by no variable; moved to its family once the client confirmed it.
Where	definitions.sas
Verification	Integrity check passes; no variable counts change.
Effect	Runs unblocked without silently altering counts.

D11 — Not-equal filters in sign slots

Attribute	Detail
Fault class	Zero-filled
Symptom	The last non-web channel variable remained zero.
How it was found	Channel-code breakdown: the code was blank on >99.9% of paid non-web orders.
Root cause	The D03 fix covered the exclusion slots only. The same bare not-equal lived in the sign slots, dropping every blank-channel row.
Fix	NULL-safe not-equal applied in the sign-slot branch of all three builders, covering every not-equal spelling. Operator literals quoted so the macro language does not parse them as operators.
Where	sql_engine.sas — sign-slot branch, three builders

Attribute	Detail
Verification	Harness run of the affected variables; emitted SQL shows (col IS NULL OR col NE val).
Effect	+1 variable (part of 469 → 475). Client confirmed blank channel = non-web.

THE FIX THAT NEEDED A FIX

The first version of this fix used a bare operator word inside a macro condition, where it was parsed as an operator. It was caught on the harness before release.

D12 — Retired product identifiers

Attribute	Detail
Fault class	Zero-filled
Symptom	Two title-promotion variables zero for every customer.
How it was found	Only ~10% of that title family's identifiers survived in the CDP cross-reference.
Root cause	Recipes filter on hard-coded legacy identifiers that were retired in migration.
Fix	Client supplied current identifiers; a legacy→current lookup was added to the bridge so recipe text stayed unchanged.
Where	compat_bridge.sas (reference-key lookup)
Verification	Values reconciled to legacy within tolerance.
Effect	+2 variables.

D13 — Activity-sparse and unported variables

Attribute	Detail
Fault class	Validation scope / missing recipe
Symptom	A one-year club count and a returner flag empty; one legacy variable absent.
How it was found	Recipe review: windows and thresholds correct; the harness audience had no recent club orders or qualifying returns. Legacy structure comparison found one unported variable.
Root cause	Two variables were correct but empty on the sample. One variable had never been ported.
Fix	The two validated on the full ~600K audience, where they populate and match legacy. The missing variable ported as one recipe on an existing builder.
Where	variables.sas (one new recipe)
Verification	Full-audience census.
Effect	+3 variables (→ 475 / 475).

D14 — Paid filter missing from a shipment count

Attribute	Detail
Fault class	Overstated
Symptom	A variable defined as paid shipments counted unpaid orders.
How it was found	Recipe compared to its own definition; probe showed ~7% of counted orders unpaid.
Root cause	Paid-amount condition omitted from the recipe's flags; sibling variables carried it.
Fix	Paid flag added to the recipe.

Attribute	Detail
Where	variables.sas
Verification	Before/after probe.
Effect	Count corrected (~7% lower). Released only after business sign-off.

4. Bridge File Specifications

4.1 compat_bridge.sas — name and join translator

Component	Purpose	Behaviour
Column mapper	Translate legacy column names to CDP names	Per-table lookup; unknown names returned unchanged
Join wrappers	Ensure both sides of every join resolve correctly	Override the engine's join builders by redefinition
Value quoting	Prevent type errors in comparisons	Quotes numeric-looking values compared with character columns
Reference-key lookup	Map retired identifiers to current ones	Client-supplied; recipe text unchanged
Dedicated recipes	Cases a name mapping cannot express	Two-field date; corrected key-domain recipe

4.2 date_bridge.sas — date semantics guard

Component	Purpose	Behaviour
Type detection	Know what the platform returned	Reads column type at run time
Guarded conversion	Correct offset for every type	Date-part extraction only where valid; DATE differenced directly
Override by redefinition	Leave the engine untouched	Loaded after the engine; replaces the date branch

4.3 Load order

1. driver.sas
2. orchestrator.sas
3. definitions.sas
4. sql_engine.sas
5. compat_bridge.sas
6. date_bridge.sas
7. variables.sas

5. Population Progression

Every change was verified before the next began, so each step in the progression can be attributed to one cause.

Stage	Change	Variables populating	Errors
Baseline	First CDP production run	344 / 475 (72.4%)	~5,000
D02	Key rewrite removed	423 / 475 (89.1%)	thousands (cascade)
D03	NULL-safe exclusions	436 / 475 (91.8%)	3
D04	Two-field date	437 / 475 (92.0%)	3
D05	Product mapping corrected	464 / 475 (97.7%)	0
Build date	Run moved to a recent build date	469 / 475 (98.7%)	0
D11–D13	Final six closed	475 / 475 (100%)	0

5.1 The build-date effect

Moving the run from an older to a recent build date populated five further variables with no code change. Every recipe looks backwards from the build date; the CDP was still receiving loads, so an older date could not reach rows loaded since. Variables with one- and two-year windows, closest to the boundary, moved first.

> **RULE**
> A variable reported empty is only empty relative to the build date it ran against. Rule out the build date and the sample scope before escalating anything as missing data.

6. Change Control

Two defects (D03 interim fix, D11 first version) were introduced by fixes, and one (D09) by a deployment. Change control was tightened in response.

Control	Practice
Edit the real file	Apply changes byte-for-byte to the source file, preserving encoding and line endings; never retype from memory
Diff every change	Diff the delivered file against the original; only the intended lines may differ
Balance check	Confirm macro definitions open and close in equal number
Fresh-session compile check	After deployment, confirm every expected macro exists in a session that has not loaded the old file
Harness first	Run the fix on the test harness across text and numeric columns and every operator spelling
One change at a time	Verify by value before layering the next change
Back up the last good output	Copy to a protected name before any risky run